Comprehensive Capstone Project Report

Production-Ready Student Management Portal

Author: Senior Software Engineering Intern

Domain

Executive Summary

Educational institutions require robust, secure, and user-friendly software solutions to streamline administrative workflows, manage student records, and enforce strict role-based access permissions. Traditional manual record-keeping methods are prone to human errors, data duplication, inefficient searching, and vulnerability to unauthorized access.

This capstone project introduces the Production-Ready Student Management Portal, a full-stack web application constructed using Python 3.10, Flask, SQLAlchemy, Flask-Login, Werkzeug Security, and Bootstrap 5. Built according to Clean Architecture and Object-Oriented Programming (OOP) paradigms, the portal provides complete end-to-end functionality for managing student records, performing multi-criteria searches, displaying real-time analytical dashboards, and enforcing strict Role-Based Access Control (RBAC).

---

1. Introduction & Project Scope

1.1 Problem Statement

Academic institutions handle thousands of student profiles containing sensitive personal details, academic metrics, and contact information. Challenges with legacy record management systems include:

1. Data
(USNs)

1.2 Objectives

The primary objective is to engineer a modular, scalable, secure, and visually appealing web application that fulfills the following goals:

- **C
utilities

---

2. Technology Stack Justification

```
Layer    Technology    Selection Rationale
```

:---

---

3. System Architecture & Design Patterns

The application adopts the Clean Architecture pattern, enforcing strict separation of concerns across five core layers:

+--------

3.1 OOP Design Principles Applied

1. Single Responsibility Principle (SRP):

- `Use

---

4. Functional Module Specifications

4.1 Authentication & Authorization Module

- **User Registration**: Password confirmation check, password hashing, unique username and email validation.

- **Us
sessio

4.2 Analytical Overview Dashboard

- Aggregates real-time metrics:

- Tota

4.3 Student Directory Management (CRUD)

- **Create Student**: Captures complete academic profile, including optional profile photo upload with UUID filename generation.

- **Re

---

5. Security Audit & Protection Mechanisms

1. Password Security: Passwords are never stored in plaintext. Passwords undergo PBKDF2 hashing with salt generation using werkzeug.security.generate_password_hash.

2. SQL
through

---

6. Verification & Automated Test Results

An automated unit test suite (test_portal.py) was executed to verify system components in an isolated in-memory testing environment:

Ran 4 t

Test Cases Verified:

1. test_

---

7. Installation & Operational Setup

1. Clone & Setup Environment:

bash

---

8. Conclusion & Future Roadmap

The Production-Ready Student Management Portal successfully satisfies all capstone requirements by offering a secure, maintainable, scalable, and intuitive software solution. The implementation adheres strictly to clean architecture principles and software engineering best practices.

Future Enhancements

- **RESTful API Layer**: Expose secure JWT-authenticated API endpoints (`/api/v1/students`).

- **CS